

AI Personal Research Agent

Complete Project Plan / PDR

Purpose: A portfolio-ready AI agent that accepts a research question, plans the research, searches the web, analyzes multiple sources, synthesizes findings, and produces a citation-backed report.

Delivery target: Working MVP first, then progressive upgrades toward a deep-research and personalized knowledge agent.

1. Project Vision

The AI Personal Research Agent is designed to behave more like a research assistant than a normal chatbot. Instead of simply generating an answer from its model knowledge, it breaks a research goal into search tasks, gathers current information, analyzes sources, identifies useful findings, and creates a structured report with citations. Personalization and memory can later be added so the agent can use the user's interests, preferences, previous research, and uploaded documents.

2. Core User Flow

User Question → Research Planner → Search Queries → Web Search → Source Collection → Source Analysis → Synthesis → Cited Research Report

3. What Makes It an Agent?

- **Goal:** The user gives a research objective rather than asking for a simple conversational response.
- **Planning:** The LLM creates multiple search queries covering different aspects of the question.
- **Tools:** The system uses a web-search API to obtain external information.
- **Reasoning:** Retrieved sources are analyzed and synthesized.
- **Iteration:** Future versions can detect information gaps and search again.
- **Memory:** Future versions can remember research history and personal preferences.

4. Architecture Options Discussed

Architecture	Flow	Use
A — Single Agent	User → LLM → Search/Read/Analyze → Report	Fastest MVP
B — Planner + Researcher + Writer	User → Planner → Researcher → Analyzer → Writer	Recommended Phase 1/2
C — Multi-Agent	Manager → Search Agent + Analyst + Fact Checker → Writer	Advanced portfolio version
D — RAG Personal Agent	Agent → Web + Personal Knowledge Base → LLM	Own PDFs/notes/history
E — Deep Research	Planner → iterative search → gap detection → verification → synthesis	Long-term version

5. Recommended Technology Stack

Layer	Technology	Reason
Frontend	Streamlit	Fast Python UI for MVP
Language	Python	Simple integration with LLM/search APIs
LLM	OpenAI API (or another available LLM)	Planning, analysis, synthesis

Layer	Technology	Reason
Web Search	Tavily	AI-oriented search and source retrieval
Agent Framework	Plain Python initially	Avoid unnecessary complexity under time pressure
Memory	SQLite / JSON initially	Simple research history
RAG later	Embeddings + vector database	Personal documents/knowledge
Deployment	Streamlit Cloud or similar	Easy prototype deployment
Version Control	Git + GitHub	Portfolio and collaboration

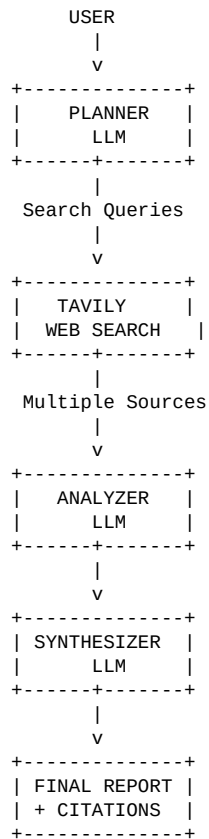
6. Phase Roadmap

Phase	Goal	Main Features
Phase 1 — MVP	Working prototype	Question → planning → web search → source analysis → report + citations
Phase 1.2	Better UX and grounding	Progress UI, source mapping, citation handling, source deduplication
Phase 2	Agentic behavior	State, tool selection, conditional re-search, information-gap detection
Phase 3	Personalization	User profile, preferences, research history, follow-up research
Phase 4	Personal Knowledge	PDF/document upload, chunking, embeddings, vector DB, RAG
Phase 5	Advanced Research	Fact checking, source credibility, contradiction detection, iterative deep research
Phase 6	Production	Authentication, persistence, deployment, monitoring, tests, security

7. Phase 1 — Detailed MVP Architecture

The first deliverable should deliberately avoid LangChain/LangGraph, vector databases, authentication, and complex multi-agent orchestration. The objective is to prove the complete research loop end-to-end in approximately one hour.

Architecture



8. Component Responsibilities

Streamlit UI: Collects the research question, starts the agent, shows progress, report, and sources.

Planner: Converts the user's question into 3–5 focused web queries.

Search Tool: Sends queries to Tavily and collects titles, URLs, and snippets/content.

Analyzer: Extracts important facts, evidence, insights, contradictions, and limitations from retrieved sources.

Synthesizer: Creates an organized report using only the collected evidence and adds source numbers.

Source Store: Keeps source metadata so report citations can be mapped back to URLs.

9. Recommended Project Structure

```
ai-personal-research-agent/
├── app.py
├── agent.py
├── search.py
├── llm.py
├── requirements.txt
├── .env
├── .gitignore
└── README.md
```

10. Phase 1 Setup Commands

```
mkdir ai-personal-research-agent
cd ai-personal-research-agent

python -m venv venv
.\venv\Scripts\Activate.ps1

New-Item app.py
New-Item agent.py
New-Item search.py
New-Item llm.py
New-Item requirements.txt
New-Item .env
New-Item README.md

pip install -r requirements.txt
```

11. Phase 1 Dependencies

```
streamlit
openai
tavily-python
python-dotenv
```

12. Environment Variables

```
OPENAI_API_KEY=your_openai_key_here
TAVILY_API_KEY=your_tavily_key_here
```

Never commit the .env file. Add venv/, .env, and __pycache__/ to .gitignore.

13. Phase 1 Search Tool

```
import os
from tavily import TavilyClient
from dotenv import load_dotenv

load_dotenv()

tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

def search_web(query, max_results=5):
    response = tavily.search(
        query=query,
        search_depth="advanced",
        max_results=max_results
    )

    results = []
    for result in response.get("results", []):
        results.append({
            "title": result.get("title", ""),
            "url": result.get("url", ""),
            "content": result.get("content", "")
        })

    return results
```

14. Phase 1 LLM Layer

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def ask_llm(prompt):
    response = client.responses.create(
        model="MODEL_AVAILABLE_TO_YOUR_API_ACCOUNT",
        input=prompt
    )
```

```
return response.output_text
```

Use a model name that is actually available to the API account. Keep the model configurable rather than hard-coding an unavailable model.

15. Agent Logic

```
def research(question):  
  
    # 1. Plan  
    plan = create_research_plan(question)  
  
    # 2. Search  
    sources = collect_sources(plan["queries"])  
  
    # 3. Analyze + synthesize  
    report = generate_report(question, sources)  
  
    return {  
        "queries": plan["queries"],  
        "sources": sources,  
        "report": report  
    }
```

Planner Prompt Design

You are an AI research planner.

The user wants to research:
{question}

Generate 3 different web search queries that explore different aspects of this question.

Return ONLY valid JSON:

```
{  
  "queries": [  
    "query 1",  
    "query 2",  
    "query 3"  
  ]  
}
```

Analysis/Synthesis Prompt Design

You are an AI research analyst.

Research Question:
{question}

Below are web sources collected by the research agent:
{source_text}

Create a research report.

Requirements:

1. Executive Summary
2. Key Findings
3. Important details
4. Disagreements/limitations where applicable
5. Do not invent facts
6. Cite factual claims using [1], [2], [3]
7. Include a Sources section
8. Return Markdown

16. Streamlit UI

```
import streamlit as st  
from agent import research  
  
st.set_page_config(  
    page_title="AI Personal Research Agent",  
    page_icon="📖",  
    layout="wide"  
)  
  
st.title("📖 AI Personal Research Agent")  
st.write(  
    "Search, analyze and synthesize information from "  
    "multiple web sources."  
)  
  
question = st.text_area(  
    "Enter your research question:"  
)
```

```

        "What do you want to research?",
        placeholder="Example: How is RAG used in modern LLM applications?"
    )

    if st.button("■ Start Research", type="primary"):

        if not question.strip():
            st.warning("Please enter a research question.")

        else:
            with st.spinner("■ Research agent is working..."):
                result = research(question)

            st.success("Research completed!")

            st.subheader("■ Research Plan")
            for query in result["queries"]:
                st.write("■", query)

            st.subheader("■ Research Report")
            st.markdown(result["report"])

            st.subheader("■ Sources")
            for i, source in enumerate(result["sources"], 1):
                st.markdown(
                    f"***[{i}] [{source['title']}]({source['url']})***"
                )

```

17. Running the MVP

```
streamlit run app.py
```

The app should open locally and provide a research-question input box. A good first test is: “What is Retrieval Augmented Generation and why is it useful for LLM applications?”

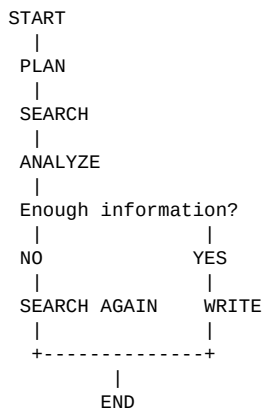
18. Phase 1 Success Criteria

- User can enter a research question.
- Planner generates multiple search queries.
- Search API returns multiple sources.
- Agent passes source content to the LLM.
- Final output contains Executive Summary, Key Findings, Conclusion, and Sources.
- Report contains source-number citations.
- The UI displays the research process and final result.
- Secrets are stored in .env and excluded from Git.

19. Phase 1.2 — Make It Look Like a Real Agent

- Add visible progress steps: Planning → Searching → Reading → Analyzing → Writing.
- Deduplicate URLs returned by different queries.
- Keep a source ID and citation map so [1] always points to the correct source.
- Limit source content to relevant material to control token usage.
- Show each source as a clickable card with title and URL.
- Add search depth controls such as Quick / Standard / Deep.

20. Phase 2 — True Agentic Loop

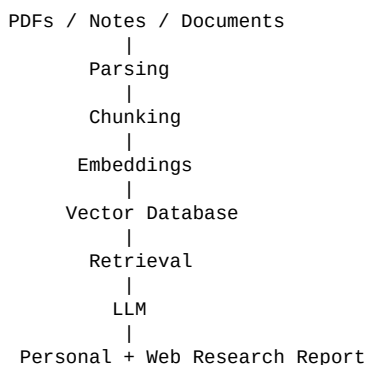


The key upgrade is conditional decision-making. After analysis, the agent evaluates whether the evidence is sufficient. If important gaps remain, it generates additional queries and searches again. This can later be implemented with LangGraph or a custom state machine.

21. Phase 3 — Personalization and Memory

- User profile: interests, preferred explanation depth, preferred source types.
- Research history: topics, reports, dates, and sources.
- Follow-up research: “Continue my research on RAG.”
- Personalized reports based on prior context.
- Saved research projects/workspaces.

22. Phase 4 — Personal Knowledge Base + RAG



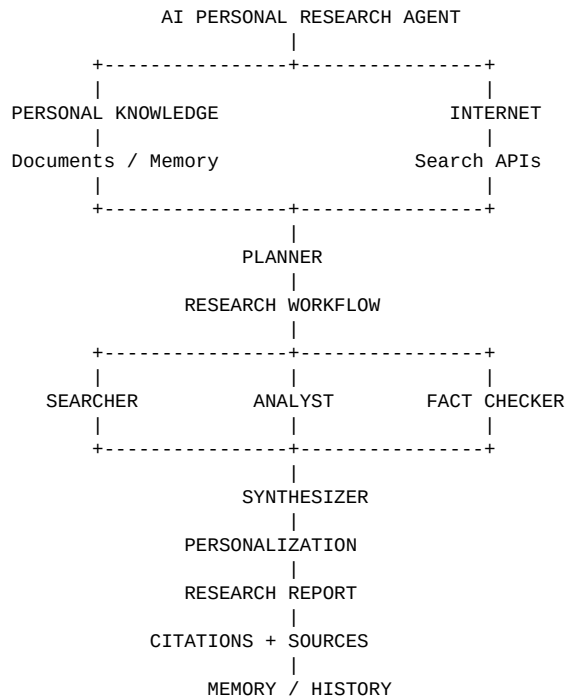
This allows the agent to combine internet research with the user's own documents. Example: “Compare what my uploaded notes say about RAG with current industry practices.”

23. Phase 5 — Advanced Deep Research

- Source credibility scoring.
- Fact verification.
- Contradiction detection.
- Claim-to-source mapping.
- Research-gap detection.
- Iterative search until evidence is sufficient.

- Specialized agents such as Searcher, Analyst, Fact Checker, and Writer.
- Export to Markdown/PDF.

24. Final Portfolio Architecture



25. Example End-to-End Execution

User: “Compare RAG and fine-tuning for building a customer-support LLM.”

- **Planner:** Creates queries covering RAG, fine-tuning, costs, latency, accuracy, maintenance, and customer-support use cases.
- **Search:** Retrieves several relevant sources.
- **Analysis:** Extracts evidence and identifies differences and limitations.
- **Synthesis:** Builds a comparison table and recommendation.
- **Output:** Cited report with source links.

26. Example Final Report Structure

```
# Research Report

## Executive Summary

## Research Question

## Key Findings

### Finding 1
...

### Finding 2
...

## Comparison

| Criterion | Option A | Option B |
|-----|-----|-----|

## Limitations / Conflicting Evidence

## Conclusion

## Sources
```

[1] Source title – URL
[2] Source title – URL
[3] Source title – URL

27. GitHub README Positioning

Project tagline: “An AI-powered personal research assistant that autonomously plans research, searches the web, analyzes multiple sources, and generates citation-backed reports.”

Suggested feature list:

- Multi-query web research
- LLM-powered research planning
- Multi-source analysis
- Citation-backed answers
- Structured research reports
- Personalization
- Research history
- Personal document RAG
- Fact checking and iterative deep research (advanced roadmap)

28. One-Hour Build Plan

Time	Task
0–10 min	Create project, virtual environment, dependencies, API keys.
10–20 min	Implement LLM planner and generate search queries.
20–35 min	Connect Tavily and collect sources.
35–45 min	Implement source analysis and report generation.
45–55 min	Build Streamlit interface and progress feedback.
55–60 min	Test, clean README, add .gitignore, push to GitHub.

29. Important Engineering Principles

- Never expose API keys in source code or GitHub.
- Ground factual claims in retrieved sources.
- Keep citations tied to actual source records.
- Deduplicate sources before synthesis.
- Control source length/token usage.
- Treat web content as untrusted input; do not blindly follow instructions contained in retrieved pages.
- Add error handling for failed searches, unavailable models, empty results, and malformed LLM JSON.
- Separate the UI, agent orchestration, search tool, and LLM layer so the project can evolve.

30. Final Development Order

1. Build Phase 1 plain-Python MVP.
2. Verify search → sources → report works.
3. Add proper citations and source deduplication.
4. Improve Streamlit UI and progress display.
5. Add an iterative research loop.
6. Introduce state/memory.
7. Add personalization.
8. Add PDF/document RAG.
9. Add fact checking and credibility scoring.
10. Package and deploy the portfolio version.

Bottom line: For the immediate prototype, the winning architecture is Planner → Search → Analyze → Synthesize → Cited Report. Build this first, prove it works, and then progressively add memory, RAG, iterative research, and multi-agent capabilities.